

Converged AI Platform

Converged AI is an open-source platform for manufacturing companies: 3D printing service bureaus, CNC shops, R&D labs, and networks of small workshops.

Its job is to take over everything that happens around production: requests, files, estimates, queues, order statuses, client communication, equipment, payments, delivery, and repeat sales. Machines still make parts, the team still makes engineering decisions, and Converged connects it all into one manageable system.

The platform includes **17 solutions** grouped into four areas: orders and clients, production and inventory, money and profit, team and accountability. These are not “modules for the sake of modules”, but ready-made scenarios for specific workshop problems.

What Converged Does

What Converged Does

A manufacturing company has two layers of work. The first is making the part, assembling the product, and completing the order. The second is everything that must happen before, during, and after production: accepting a request, not losing the file, agreeing on the price, putting the job into a queue, warning the client, understanding machine load, spotting delays, and getting paid.

Converged covers that second layer. It is a digital control layer that connects the website, client requests, internal tasks, equipment, inventory, payments, notifications, and analytics. The owner sees not a pile of disconnected tools, but a live picture of the business: what came in, what was accepted, what is queued, what is ready, where a delay is likely, and where margin is being lost.

The platform is especially useful when production already works, but management depends on chats, spreadsheets, employee memory, and manual control. Converged does not force a company to adopt a heavy ERP or MES immediately. It starts with concrete processes and gradually builds a unified system around them.

In a typical scenario, a client submits a request through a website, form, messenger, or operator. Converged stores the request, attaches files, creates an order, helps estimate the price, places the work into the queue, tracks execution, and keeps the client informed. An operator can ask for status in the interface or through the AI chat, and the system answers from real data rather than guesses.

As a result, the workshop depends less on manual coordination. People focus on production and decisions that really require expertise, while the platform handles routing, deadline control, repeated messages, status collection, and action preparation.

Solution System

Solution System

Converged is not sold as an empty platform where the customer must first invent the architecture and assemble modules. The basic unit of value is a **solution**: a ready-made working scenario that closes one clear business problem.

The platform provides **17 solutions** grouped into four areas:

- **Orders and clients** — incoming requests, service showcase, client history, statuses, communication, and repeat sales.
- **Production and inventory** — equipment load, queues, materials, quality control, failures, and shipments.
- **Money and profit** — cost, margin, payments, receivables, pricing, and growth scenarios.
- **Team and accountability** — responsibility zones, shifts, standards, knowledge base, and onboarding.

A solution inside Converged is not just a screen in the interface. It usually includes a data model, workflow, roles, notifications, AI-agent actions, and integrations with equipment or external services. The owner chooses not a “feature”, but a problem: speed up request handling, see the machine queue, understand order profitability, or bring order to shifts.

The detailed description of all solutions belongs in a separate section. In the product documentation, the important point is the principle: Converged AI is the platform, and solutions are applied scenarios that run inside it and gradually close different areas of a manufacturing business.

Equipment and Shop Floor

Equipment and Shop Floor

Converged does not replace machine firmware and does not try to control production blindly. It connects above the equipment, reads telemetry, links machine state to orders, and shows what is really happening on the shop floor.

The platform is designed for different equipment types: Bambu Lab, Marlin, and Klipper 3D printers, CNC machines, robotic cells, and specialized adapters. Where possible, Converged receives statuses, errors, execution progress, temperature, task queues, and other technical data. Where direct control is risky or unavailable, the system remains a layer of observation and coordination.

For the owner, this means one simple thing: equipment stops being a set of separate windows. You can see which machines are busy, where there is idle time, what is late, which order is tied to a specific operation, and where an operator needs to intervene. As the fleet grows, this visibility becomes more important than manually switching between separate printer or machine interfaces.

Converged also connects equipment to the business process. A task is not merely “printing” or “milling”; it sits in the context of an order, deadline, client, material, payment, and next step. The shop floor becomes part of the shared system instead of a separate island.

Processes

Processes

The main problem of a growing workshop is rarely the lack of one more button. More often, processes live in people's heads: who must answer the client, when to calculate the price, who checks the file, when production starts, whom to notify about a delay, and what happens after shipment.

In Converged, these chains are described as workflows. A typical process can go from request to estimation, approval, queueing, production, quality control, payment, delivery, and notifications. The user usually does not build a graph from scratch: ready-made scenarios ship with the solutions, and configuration comes down to rules, roles, deadlines, integrations, and notifications.

Technically, execution is moved into the Runtime layer. It runs workflows, cron jobs, integration steps, and business logic while remaining stateless: persistent data stays in microservices, and Runtime is responsible for executing chains. This keeps business logic from being scattered across dozens of services and leaves one clear place where process rules live.

For complex deployments, workflows can be extended. A developer describes scenarios as typed TypeScript classes, and AI agents can launch permitted actions inside those scenarios. But for a regular user, the goal is different: not to build a constructor, but to enable a ready process and get a managed result.

AI Layer

AI Layer

AI in Converged is not a separate chat added for appearance. It is a control layer over data, interface, and processes. A user can ask what is happening with an order, request a client reply, find a delay, launch an allowed workflow, or collect a summary of machine load.

The model receives context from the platform: requests, statuses, files, telemetry, client history, access rights, and current tasks. The answer is therefore based on company data, not generic reasoning. If an action is needed, AI does not bypass the system directly: it calls allowed functions, microservices, or workflows through a controlled layer.

Model providers connect through adapters. A single installation can use GPT, Claude, DeepSeek, Mistral, Gemini, or other engines if they fit the task and the client's policy. One model can communicate with clients, another can parse technical requirements, and a third can analyze documents or production statuses.

The key principle is control. AI has its own access profile, every action is logged, and critical operations run under the same rights and policies as human actions. This allows agents to handle routine work without giving them uncontrolled power over production.

Architecture

Architecture

Converged is designed as a modular platform, but not as a chaotic collection of microservices. The separation is simple: the interface shows data and launches actions, Runtime executes processes, microservices own data, and adapters connect equipment and external systems.

User / client ↓ **UI and micro-frontends** ↓ **Runtime:**
workflows, cron, integrations, AI actions ↓ **Microservices: typed**
APIs and owned data ↓ **Storage / Behemoth / files / SQL / KV /**
metrics ↓ **Equipment, messengers, payment and external services**

Microservices intentionally stay thin. Each service is responsible for its data area, validation, and typed API. It should not know the internal logic of neighboring services and should not become a hidden center of business processes. This lowers coupling and makes the system easier to maintain.

All cross-domain logic is moved into Runtime. If the system needs to accept an order, query several services, create a task, send a notification, wait for an event, and update status, that is executed in a workflow. Runtime does not store persistent state itself: it writes history, variables, and results through the services that own their storage.

Storage is built around isolation. Instead of one shared database, each domain gets its own data boundaries: SQL, key-value, file storage, column data, vector indexes, or graph relations where needed. This approach helps move workspaces, limit access, and avoid a common database where different clients' data is mixed.

The frontend is modular as well. The shared shell loads independent micro-frontends through an import map, so individual interface areas can evolve without rebuilding the entire product. For the user it remains one system, while development keeps clear areas of responsibility.

Deployment

Deployment

Converged supports several installation scenarios: from a small workshop to a production deployment in a company's infrastructure. The base platform runs on **k3s**, a lightweight Kubernetes distribution suitable for edge devices, local servers, and cloud environments.

There are two main profiles:

- **Mono** — UI, Runtime, microservices, storage, and cache are packed compactly. This mode is for development, prototypes, demos, and small installations where simple startup matters most.
- **Multi** — UI, Runtime groups, domain groups of microservices, storage, and cache are separated. This is the standard production profile when isolation, scaling, and more precise load control are needed.

Both profiles use the same code. Only the container topology and configuration differ. A company can start with a compact installation and later move the same system into more serious infrastructure without rewriting the product.

In self-hosted scenarios, the client controls installation, networking, backups, updates, and the physical location of data. This fits companies with internal security requirements or a desire to keep production fully on their side. The cloud delivery removes operational work: the platform is deployed and updated by the service team, while the client receives a ready working environment.

A hybrid option is also possible: sensitive data and equipment stay local, while the cloud is used for updates, external access, coordination of distributed teams, or selected AI functions. The key principle is not to lock the client into a single delivery model.

Technologies

Technologies

The server side of Converged is built on **Bun** and **Elysia**. Bun starts JavaScript and TypeScript quickly, uses memory efficiently, and fits compact edge deployments. Elysia is used as the HTTP layer for backend plugins and microservices.

Service contracts are described with types. NRPC binds TypeScript interfaces to implementations and generates client packages, so the frontend, Runtime, and backend work with the same contracts instead of disconnected string-based APIs.

Data storage uses a set of lightweight stores for different tasks: SQL, key-value, files, column data, vector indexes, and graph relations. The native Behemoth layer and Zig adapters cover tasks where low overhead, equipment access, Unix sockets, or FFI matter.

The frontend is a React platform with micro-frontends. The shared shell loads separate UI modules, and product scenarios can evolve independently. This matters for a platform with many solutions: the interface should not become one heavy monolith.

Orchestration and delivery are built around k3s, Helm, and configuration profiles. The same component set can be assembled into a compact mono profile or split into groups for production.

Performance

Performance

Converged is designed for production sites that do not always have a large server fleet. The system therefore avoids unnecessary weight: Bun reduces backend process overhead, Runtime stays stateless, and microservices can be grouped by load type instead of running hundreds of separate containers.

Performance comes from architecture, not from one trick. Data does not pass through unnecessary layers, services own their stores, Runtime parallelizes workflows and cron jobs, and native adapters are used where HTTP or a regular JS layer would add too much overhead.

A compact installation can run on a small server or single-board computer if the workload matches the scale of the workshop. As the company grows, Runtime, microservices, and storage groups can be separated to use more CPU cores, isolate heavy tasks, and prevent one bottleneck from stopping the whole system.

The platform does not promise infinite performance “out of the box”. Bottlenecks depend on equipment, file volume, order count, AI providers, and integrations. Converged’s architecture allows a company to start compactly and scale only the parts that actually become hot.

Security

Security

Converged starts from the assumption that production data should not be thrown into one shared pile. Orders, client files, technological parameters, payments, messages, and equipment telemetry must be separated by workspaces and responsibility zones.

Architecturally, this is supported by data isolation. Microservices own their stores, and workspaces can have separate directories, keys, files, and access boundaries. This simplifies export, self-hosted migration, backups, and audit.

Access rights apply not only to people, but also to AI agents. If a model launches an action, reads data, or calls a workflow, it must happen within its permission profile. Actions are logged, so it is possible to reconstruct who or which agent initiated a step, what data was affected, and how the scenario ended.

Self-hosted and private deployments give the client full control over infrastructure: network, secrets, API keys, backups, and physical data location. Cloud mode is operationally easier, but should not become vendor lock-in: data must remain portable, and scenarios must remain reproducible in another installation.

Licensing

Licensing

Converged is distributed under **AGPL-3.0**. This is a copyleft license for network software: if you modify the platform and provide access to it over a network, the changes must be disclosed according to the license terms.

For users, this means the self-hosted version can be deployed without buying a license for the code itself. This path fits companies that need control, local installation, auditability, or experiments on their own hardware. Operational responsibility remains with the installation owner: updates, backups, monitoring, security, and availability.

Cloud delivery is not the sale of closed code, but a service around an open-source platform. The client pays for fast launch, support, updates, backups, monitoring, infrastructure, and predictable operation. For many workshops, this is cheaper than building DevOps competence internally.

This balance matters for trust: the core remains open, the community can inspect and improve the platform, and the commercial model is built around implementation, support, and controlled value delivery.

Community

Community

Converged is designed as an open manufacturing platform, not a closed SaaS box. The core is available under an open-source license, and integrations, microservices, micro-frontends, workflows, and applied solutions can grow around it.

This matters in manufacturing: different workshops use different equipment, materials, quality standards, and supply chains. A closed system quickly hits the limits of a single vendor. An open architecture allows adapters and scenarios to be added for real operating conditions.

An extension can be simple: a new equipment adapter, payment-service integration, import from an old website, a separate UI module, or a workflow for a specific industry. If an extension is useful to other participants, it can become part of the solution catalog or remain a private implementation.

Open does not mean uncontrolled. Extensions must pass technical review, respect architectural boundaries, and not receive broader data access than they need. Trust in the ecosystem is built on source code, reproducibility, and a clear permission model.

Source Code

Source Code

The project is developed openly. Source code, current architecture, and working materials are available in the repository:

<https://github.com/solenopsys/converged>

The repository is useful not only for developers. It shows how the microservices, Runtime, micro-frontends, contract generation, deployment profiles, and hardware adapters are organized. For companies considering self-hosted or private deployment, this is an important part of evaluation: the platform is not a closed box.